

Online Complaint Registration and Management System

Phase 3: Project Design Phase

SmartBridge Internship Project Submission

Team: Online Complaint Registration and Management System

3.1 Problem-Solution Fit

Problem	How the Solution Addresses It
No centralized place to register a complaint	A single web portal (React frontend + REST API) available to all users, replacing scattered phone/email/in-person channels.
No visibility into complaint status	A role-aware dashboard shows real-time status: Pending → Assigned → In Progress → Resolved → Closed.
No accountability for who is handling an issue	Complaints are explicitly assigned to an agent by the admin, and the assignment is visible to the user.
Users unaware when something changes	A notification is automatically generated on complaint creation, agent assignment, status change, and resolution.
No insight for the organization into complaint trends	Admin analytics dashboard aggregates totals, pending/resolved counts, and category/status breakdowns.

3.2 Proposed Solution

Parameter	Description
Problem Statement	Citizens/customers lack a centralized, transparent way to register and track complaints, resulting in lost records and no accountability.
Proposed Solution	A full-stack MERN web application — Online Complaint Registration and Management System — enabling complaint registration, real-time tracking, agent assignment, and admin oversight.
Uniqueness / USP	Combines role-based workflow (User / Agent / Admin) with automatic notifications and an analytics dashboard in a single lightweight web app, without requiring a native mobile install.
Target Users	Citizens or customers raising complaints; support agents resolving them; administrators overseeing the process.
Scalability	Stateless JWT auth and a cloud-hosted MongoDB Atlas database allow the backend and frontend to scale independently on Render.

3.3 Solution Architecture

The system follows a three-tier client-server architecture: a React single-page application (client tier), an Express.js REST API secured with JWT-based middleware (application tier), and a MongoDB Atlas database (data tier). The frontend and backend are deployed as independent services on Render, communicating over HTTPS.

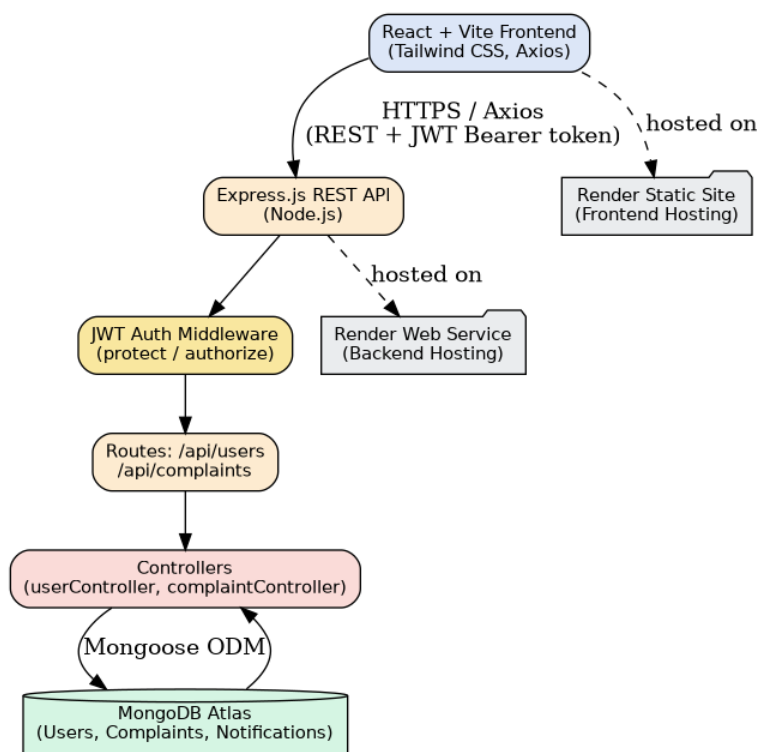


Figure 3.1 — Solution / System Architecture

3.3.1 Key Architectural Decisions

- Stateless authentication via JWT — no server-side session storage, enabling horizontal scaling of the API.
- Role-based authorization enforced centrally in Express middleware (protect, authorize), rather than duplicated per route.
- Separation of concerns: routes define endpoints, controllers hold business logic, models define schema/validation — a standard MVC-style layout for maintainability.
- Frontend and backend deployed and scaled independently (Render Static Site + Render Web Service), connected via a configurable VITE_API_URL environment variable.